

提高级 CSP-S 第 9 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 下列选项中最大的数是()。
A. $(504)_{10}$ B. $(111111000)_2$ C. $(770)_8$ D. $(1FA)_{16}$
- 主机 IP 地址为 194.32.6.22,掩码为 255.255.255.192,子网地址是()。
A. 194.32.6.22 B. 194.32.0.0 C. 0.0.0.22 D. 194.32.6.0
- 现代普通计算机的网关不能设置为()。
A. 10.16.13.100 B. 127.0.0.1 C. 234.123.6.5 D. 168.10.7.100
- 2021 个点的二叉树的叶子个数最大是()。
A. 1010 B. 1 C. 2020 D. 1011
- 深度优先搜索时,一定需要用到的数据结构是()。
A. 栈 B. 二叉树 C. 链式前向星 D. 队列
- 2023 的 2023 次方对 5 取模的结果为()。
A. 1 B. 2 C. 3 D. 4
- 众所周知,希尔排序的复杂度与增量序列有着很密切的联系,下列增量序列中,()在确保正确的前提下会使希尔排序的最坏时间复杂度最好。
A. $\{1,2,4,8,\dots\}$ B. $\{5,19,41,109,\dots\}$
C. $\{1,3,7,\dots,2^k-1\}$ D. $\{1,2,3,4,5,\dots\}$
- NP 问题的讨论是计算机科学中一个重要的话题。以下问题中,()在一般情况下有着关于输入规模 n 的多项式复杂度的确定性正确算法。
A. 0-1 背包问题, n 为物品个数 B. 数字的质因数分解, n 为数字位数
C. 旅行商问题, n 为点的个数 D. 求矩阵行列式, n 为矩阵的行数
- 表达式 $3 * (1-2) + 4$ 的后缀形式为()。
A. 3 1 2- * 4+ B. 3 1-2 * 4+
C. + * 3-1 2 4 D. 3 1 * 2-4+
- 只采用路径压缩的并查集最坏平均复杂度为()。
A. $O(n)$ B. $O(n\alpha(n))$ C. $O(n \log n)$ D. $O(n^2)$
- 有以下程序段:
const int N=1000, M=10000; int h[N], tot, n, m;
struct edge { double c; int nxt, v; } e[M<<1]; double d[N];
其占用的静态空间大小约为()。
A. 30000Bytes B. 324KB C. 3200KB D. 3MB
- 下列表达式中,不论变量 p, q 如何取值,恒为真的只有()。
(1) $((!p) \&\& q) || (p \&\& q)$
(2) $((!p) \&\& q) \&\& (p || q)$
(3) $(((!p) || q) \&\& (p || q)) || p || (!q)$
(4) $(p \&\& (!q) \&\& q) || (((!p) || q) \&\& (p || q))$
A. (3)(4) B. (1)(3) C. (2)(4) D. (3)
- 定义一个数是“好的”:当且仅当这个数是个六位数(允许有前导零),并且里面含有数字 9。“好的”数的个数是()。
A. 1000000 B. 531441 C. 468559 D. 999999
- 现在有 20 个人约定一起玩一局游戏。但是因为可能要补作业,所以每个人有 50% 的概率最终参加。他们认为一局游戏的有趣程度为参加人数的平方,则游戏有趣程度的期望为()。

A. 425/4 B. 105 C. 110 D. 100

15. 以下()是 C++之父?

- A. 本贾尼·斯特劳斯特卢普(Bjarne Stroustrup)
- B. 图灵(Alan Turing)
- C. 吉多·范罗苏姆(Guido van Rossum)
- D. 姚期智

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特殊说明外,判断题 1.5 分,选择题 3 分,共计 40 分)

1.

```

01 #include<algorithm>
02 #include<iostream>
03 using namespace std;
04 const int MAXN=1000;
05 int a[MAXN],c[MAXN];
06
07 int main() {
08     int n,x,y;
09     cin>>n>>x>>y;
10     int sum=(x ^ y)+((x & y)<<1);
11     n=n<<3 ^ 5 & 3 | 2;
12     for (int i=1;i<=n;++i)
13         cin>>a[i];
14     int answer;
15     answer=c[1]=max(a[1],0);
16     for (int i=2;i<=n;++i) {
17         c[i]=max(c[i-1]+a[i],a[i]);
18         if (c[i]<=0)
19             c[i]=0;
20         answer=max(answer,c[i]);
21     }
22     if (answer<=sum)
23         puts("Good.");
24     else
25         puts("It ' s too sad.");
26     return 0;
27 }
```

●判断题

- (1) $sum=x+y$ 。()
- (2) 将程序 18,19 行删去后输出结果不变。()
- (3) 14 行 `answer` 没赋初值可能会导致答案错误。()
- (4) 输入的 n 应小于等于 125,否则会运行时错误。()

●选择题

- (5) 若输入的 $n=15$,则程序 12 行以后的 n 的值为()。
 - A. 60 B. 123 C. 3 D. 30
- (6) 若 12 行时 $n=11$,且输入 a 数组元素依次是:5,12,-7,-11,-9,4,9,5,-6,11,-1,则程序运行到 22 行时 `answer` 是()。
 - A. 23 B. 18 C. 17 D. 46

2.

```

01 #include<cmath>
02 #include<cstring>
03 #include<iostream>
04
05 const int MAXN=1000000;
06 int n;
07 int f1[MAXN],f2[MAXN],f3[MAXN];
08
09 int calc_f1(int x) {
10     int ans=1;
11     int maxNum=sqrt(x);
12     for (int i=2;i<=maxNum;++i)
13         if(x%i==0) {
14             ans=-ans;
15             x /=i;
16             if (x%i==0)
17                 return 0;
18         }
19     if (x !=1)
20         ans=-ans;
21     return ans;
22 }
23
24 int calc_f2(int x) {
25     return x;
26 }
27
28 void convolute() {
29     memset(f3,0,sizeof(f3));
30     for (int i=1;i<=n;++i)
31         for (int j=1;j<=n/i;++j)
32             if (i*j<=n)
33                 f3[i*j]=f3[i*j]+f1[i]*f2[j];
34 }
35 int main() {
36     scanf("%d",&n);
37     for (int i=1;i<=n;++i)
38         f1[i]=calc_f1(i);
39     for (int i=1;i<=n;++i)
40         f2[i]=calc_f2(i);
41     convolute();
42     printf("%d\\backslashn",f3[n]);
43     return 0;
44 }

```

●判断题

- (1) 若将程序第 31 行的 n/i 改为 n 后运行,且已知程序可以安全地运行至第 36 行而不发生任何未定义行为,则程序依旧可以正常输出正确的答案。()
- (2) 去掉程序第 03 行,程序依然可以正常编译。()

●选择题

- (3) 该程序中,第 30~33 行运行的时间复杂度为()。
- A. $O(n^2)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n \sqrt{n})$
- (4) 若输入 3,程序的输出为()。
- A. 2 B. 1 C. -1 D. 发生未定义行为
- (5) 若输入 1003,程序的输出为()。
- A. 1002 B. 928 C. 1 D. 发生未定义行为
- (6) 若输入 1000000,程序输出为()。
- A. 999999 B. 400000 C. 0 D. 发生未定义行为

注:本题中你只需要考虑数组越界这一种未定义行为。

3.

```

01 #include<iostream>
02 using namespace std;
03
04 const int maxn=10000000;
05 int n,size;
06 int prime[maxn+5];
07 bool vis[maxn+5];
08
09 int main() {
10     cin>>n;
11     size=0;
12     for (int i=2;i<=n;++i) {
13         if (!vis[i]) {
14             size=size+1;
15             prime[size]=i;
16         }
17         for (int j=1;i*prime[j]<=n;++j) {
18             vis[i*prime[j]]=1;
19             if (i%prime[j]==0)
20                 break;
21         }
22     }
23     int sum=0;
24     for (int i=1;i<=size;++i)
25         sum=sum+prime[i];
26     cout<<sum<<endl;
27     return 0;
28 }

```

●判断题

- (1) n 必须小于等于 10000004,否则程序可能会运行时错误。()
- (2) 输出可能为 1。()

(3) 若输入为 10000000, 那么程序会输出小于等于 10000000 的质数之和。()

(4) 该程序的输出含义是小于等于 n 的所有质数之和。()

● 选择题

(5) 若输入 n 为 40, 那么输出为()。

A. 820 B. 197 C. 196 D. 217

(6) (3.5 分) 若输出的答案在 500000~500000000 之间, 那么 n 的量级约为()。

A. 100~1000 B. 1000~10000 C. 10000~100000 D. 100000~1000000

(7) (3.5 分) 该程序的复杂度为()。

A. $O(n)$ B. $O(n^2)$ C. $O(n \log n)$ D. $O(n \log \log n)$

三、完善程序(单选题, 每小题 3 分, 共计 30 分)

1. (背包问题 1) 有一个质量上限为 m 的背包和 n 个物品, 物品分三类, 第一类物品有取数量限制, 第二类物品只能取一个, 第三类物品可以无限取。

第 i 个物品的类别为 $tp[i]$ ($tp[i] \in [1, 2, 3]$) 质量为 $w[i]$, 价值为 $c[i]$, 如果 $tp[i] = 1$, 还会有一个 $a[i]$ 表示这种物品最多只能取 $a[i]$ 个。求这个背包可以装下的最大价值。

$M \leq 1000, n \leq 100, w[i], c[i] \leq 1000, a[i] \leq 10, 1 \leq tp[i] \leq 3$ 。

试补全程序。

```
01 #include<iostream>
02 using namespace std;
03 int n, m, ans;
04 int f[1001], a[101], w[101], c[101];
05
06 int main() {
07     cin >> m >> n;
08     for (int i = 1; i <= n; ++i) {
09         cin >> tp[i] >> w[i] >> c[i];
10         if (tp[i] == 1) cin >> a[i];
11     }
12     f[0] = 0;
13     for (int i = 1; i <= m; ++i)
14         f[i] = ①;
15     for (int i = 1; i <= n; ++i) {
16         if (tp[i] == 1) {
17             for (int j = 1; j <= a[i]; ++j)
18                 for (int v = m; v >= w[i]; --v)
19                     f[v] = max(f[v], ⑤);
20         }
21         else if (tp[i] == ②) {
22             for (int v = m; v >= ③; --v)
23                 f[v] = max(f[v], ⑤);
24         }
25         else {
26             for (④)
27                 f[v] = max(f[v], ⑤);
28         }
29     }
```

```

30     printf("%d", f[m]);
31     return 0;
32 }

```

(1) ①处应填()。

A. 0 B. 1 C. -1 D. 2147483647

(2) ②处应填()。

A. 2 B. 0 C. 3 D. 1

(3) ③处应填()。

A. m B. 0 C. w[i] D. 1

(4) ④处应填()。

A. int v=0; v<=m; ++v B. int v=w[i]; v<=m; ++v

C. int v=m; v>=w[i]; --v D. int v=m; v>=0; --v

(5) 三个⑤处是同一个代码,应填()。

A. w[i] B. f[v-w[i]]+c[i]

C. f[v-w[i]] D. f[v-c[i]]+w[i]

2. (背包问题 2) 有 N ($0 < N \leq 50$) 个物品,第 i 个物品体积为 $v[i]$ ($0 \leq v[i] \leq 10^{18}$), 价值为 $w[i]$ ($0 \leq w[i] \leq 10^{17}$)。现有一个容量为 V ($0 \leq V \leq 10^{18}$) 的背包,你需要选择一些物品使得体积之和不超过 V 且价值最大。求出最大价值及方案。如有多种方案,尽可能选择编号较大的方案(即尽量选第 n 个,仍有多组解选第 $n-1$ 个,以此类推)。

提示:将物品按标号是否大于 $n/2$ 分成两组,每组内枚举如何选择,再在两组间确定方案。

试补全程序。

```

01 #include<iostream>
02 using namespace std;
03
04 const long long inf= ①;
05 long long v[41];
06 long long w[41];
07
08 struct Item{
09     long long value,weight,choose;
10 } itm[2][1<<20];
11
12 bool cmp1(Item a,Item b) {
13     return a.value<b.value || (a.value==b.value && a.choose<b.
choose);
14 }
15 bool cmp2(Item a,Item b) {
16     return a.value<b.value || (a.value==b.value && a.choose==
b.choose);
17 }
18 int get_id(int n) {
19     for (int i=0;;++i)
20         if (!(n>>i)) return i-1;
21 }
22

```

```

23 void sort(Item item[],int L,int R) {
24     if (L>=R) return;
25     int x=rand()%(R-L+1)+L;
26     swap(item[L],item[x]);
27     int a=L+1,b=R;
28     while (a<b) {
29         while (a<b && cmp1(item[a],item[L]))
30             ++a;
31         while (a<b && ②)
32             --b;
33         swap(item[a],item[b]);
34     }
35     while (b<=R && item[b].value==item[L].value)
36         ++b;
37     if (item[a].value<item[L].value)
38         swap(item[a],item[L]);
39     sort(item,L,a-1);
40     sort(item,b,R);
41 }
42
43 int solve(int L,int R,Item item[]) {
44     item[0].value=0;
45     item[0].choose=0;
46     int len=③;
47     for (int i=1;i<=len;++i) {
48         int tmp=④;
49         item[i].value=item[i-tmp].value+v[L+get_id(tmp)];
50         item[i].weight=item[i-tmp].weight+w[L+get_id(tmp)];
51         if (item[i].value>inf) item[i].value=inf;
52         item[i].choose=i;
53     }
54     sort(item,0,len);
55     return len;
56 }
57
58 int main() {
59     int N;
60     long long V,ans=0;
61     cin>>N>>V;
62     for (int i=1;i<=N;++i)
63         cin>>v[i]>>w[i];
64     int len1=solve(1,N/2,itm[0]);
65     int len2=solve(N/2+1,N,itm[1]);
66     for (int i=1;i<=len2;++i)
67         itm[1][i].value=max(itm[1][i].value,itm[1][i-1].value);
68     long long choose=0;

```



```

69     for (int i=0;i<=len1;++i) {
70         while (len2>=0 && itm[0][i].value+itm[1][len2].value>V)
71             --len2;
72         if (len2>=0) {
73             long long new_value = itm[0][i].weight + itm[1][len2].
weight;
74             long long new_choose= ⑤;
75             if (ans==new_value)
76                 choose=max(choose,new_choose);
77             if (ans<new_value) {
78                 ans=new_value;
79                 choose=new_choose;
80             }
81         }
82     }
83     cout<<ans<<endl;
84     for (int i=0;i<N;i++)
85         if (choose & (1LL<<i))
86             cout<<i+1<<' ';
87     cout<<endl;
88 }

```

(1) ① 处应填()。

A. 100000000000000000LL

B. 0x3fffffffffffffLL

C. 0x7fffffffffffffLL

D. 0xfffffffffffffLL

(2) ② 处应填()。

A. cmp1(item[L], item[b])

B. cmp1(item[b], item[L])

C. cmp2(item[L], item[b])

D. cmp2(item[b], item[L])

(3) ③ 处应填()。

A. (1<<R-L+1)-1

B. 1<<R-L+1

C. (1<<R)-1

D. 1<<R

(4) ④ 处应填()。

A. i

B. i & -i

C. i+(i & -i)

D. i-(i & -i)

(5) ⑤ 处应填()。

A. itm[0][i].choose | itm[1][len2].choose

B. itm[0][i].choose | (itm[1][len2].choose<<N/2LL)

C. itm[0][i].choose | (itm[1][len2].choose<<N/2)

D. itm[0][i].choose | (itm[1][len2].choose<<N)